Полный конспект-гайд для сдачи Лабораторной работы №2

Дисциплина: Нереляционные и распределенные базы данных (nosql_db)
Преподаватель: ас. Жишкович Андрей Владимирович | Тема: Инсталляция и основы работы с MongoDB
Студент: Абдулханов Абдул-Рахим (ФеП-33) | Львов-2026

1. Главный смысл и общая картина лабораторной работы (Big Picture)

Зачем вообще нужна эта лабораторная работа: это фундамент и мостик перехода от реляционных баз данных (SQL) к документо-ориентированному подходу NoSQL. В традиционном SQL любая сущность дробится на плоские таблицы и строки, а потом долго и дорого соединяется через JOIN. В MongoDB одна сущность (товар, клиент, заказ) хранится целиком как одна законченная папка-карточка в формате JSON/BSON. Лабораторная работа №2 посвящена тому, чтобы развернуть инструменты и научиться правильно мыслить такими документами, не пытаясь строить привычные таблицы.

Вся работа строго соответствует 4 этапам методички:

- **1.** Этап 1 (Ознакомление): изучение документо-ориентированной модели, архитектуры MongoDB и формата JSON по лекциям преподавателя.
- **2.** Этап 2 (Инсталляция): установка сервера mongod, консольного шелла mongosh и графического интерфейса Compass. Настройка системной переменной PATH и проверка версии 'mongosh --version' в командной строке.
- **3.** Этап 3 (Работа с открытыми данными): скачивание реального открытого набора данных Нобелевского комитета (nobel_prizes.json) и анализ его древовидной структуры через онлайн-валидатор.
- **4.** Этап 4 (Собственное проектирование и валидация): выбор предметной области магазина TechStore, проектирование двух коллекций (products и orders), составление 6 документов с обязательными массивами и вложенными документами и проверка синтаксиса каждого документа в валидаторе.

> **Формулировка ответа на защите на вопрос «Что было сделано в лабе?»:**
> «Лабораторна робота №2 є ввідною. Я встановив повний локальний стек MongoDB (сервер mongod, консоль mongosh та клієнт Compass), налаштував системний PATH і перевірив версію. Дослідив структуру реального ієрархічного файлу Nobel Prize. Спроєктував дві колекції для інтернет-магазину TechStore: каталог товарів з масивами тегів і вкладеними характеристиками та журнал замовлень з вкладеною доставкою. Виконав синтаксичну валідацію кожного з 6 документів онлайн-валідатором на відповідність стандарту RFC 8259».

2. Иерархия NoSQL и сравнение с SQL на простых аналогиях

Полная иерархия сущностей NoSQL от общего к частному выглядит так: Кластер (Cluster) -> База данных (Database) -> Коллекция (Collection) -> Документ (Document) -> Поле (Field).

Чтобы не путаться в терминах, запомни простую бытовую аналогию: Кластер - это вся сеть филиалов компании. База данных - это Шкаф в конкретном филиале. Коллекция - это Полка в шкафу. Документ - это Папка на полке. Поле - это конкретная строчка в документе.

MongoDB в порівнянні з реляційною базою даних

MongoDB	RDBMS
Нереляційна база даних	Реляційна база даних
Документно-орієнтована	Рядково-орієнтована
Поле-орієнтована	Стовпцево-орієнтована
Колекційно-орієнтована	Таблично-орієнтована
Клієнт на основі JavaScript для запитів	Вбудованого клієнта немає
Майже не піддається SQL-ін'єкціям	Легко піддається SQL-ін'єкціям
Має динамічну схему	Попередньо визначена статична схема
Легко зберігати ієрархічні дані	Важко зберігати ієрархічні дані
Горизонтально масштабована	Можливе вертикальне масштабування

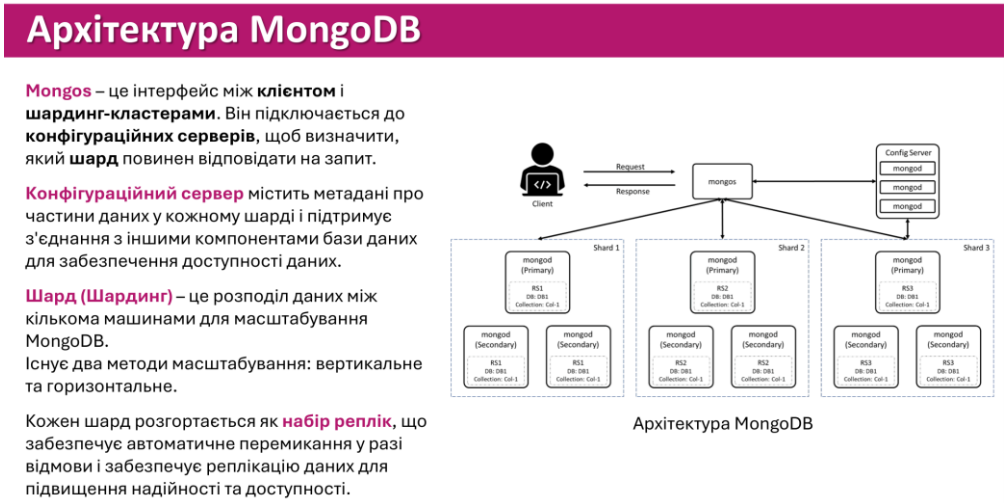
Рис. 1. Сопоставление понятий реляционных баз данных и MongoDB (Слайд 9 лекции Жишковича А. В.)

SQL (Таблицы / Строки)	MongoDB (Коллекции / Документы)	Суть на простых пальцах
База данных (Database)	База данных (Database)	Общий шкаф проекта. Хранит группу тематических коллекций.
Таблица (Table)	Коллекция (Collection)	Полка в шкафу. В SQL все строки обязаны быть под одну гребенку, в Mongo на одной полке лежат гибкие документы.
Строка / Запись (Row)	Документ (Document)	Отдельная папка-карточка. Внутри данные в виде пар 'ключ: значение'.
Столбец (Column)	Поле (Field)	Атрибут карточки: строка, число, дата, целый массив или вложенный поддокумент.
JOIN (соединение)	Вложенность (Embedded Docs)	В SQL таблицы связывают внешними ключами. В Mongo всю информацию сразу кладут внутрь одного документа.
Primary Key (ID)	Поле _id (ObjectId)	Уникальный паспорт документа на 12 байт, чтобы отличать карточки друг от друга.
Схема (CREATE TABLE)	Динамическая схема (Schemaless)	В SQL схема фиксируется до записи. В Mongo каждый документ сам несет свои поля (Schema-on-Read).

Ключевой теоретический вопрос: «Где живут коллекции - в базе данных или в шарде?»
Ответ: Логически коллекция принадлежит базе данных (шкафу). А физически, если включен шардинг на больших серверах, ОДНА И ТА ЖЕ коллекция разрезается на порции (чанки) и раскидывается по разным физическим серверам-шардам. Для программиста это единая коллекция, а для винчестеров - файлы на разных компьютерах.

3. Архитектура шардинга и масштабирования (Слайд 10 лекции)

Когда интернет-магазин вырастает и объем товаров с заказами переваливает за терабайты, один физический компьютер не справляется с нагрузкой. MongoDB масштабируется горизонтально (Scale-Out) через шардинг:



Архітектура MongoDB

Рис. 2. Компоненты распределенного кластера MongoDB: mongos, Config Servers, Shards (Слайд 10 лекции)

- **1. Склады данных (Shards) -** Шард (Shard): это отдельные склады с данными. Например, товары A-M лежат на Шарде 1, а N-Z - на Шарде 2. Внимание: шард - это не один компьютер, а набор реплик (Replica Set, минимум 3 машины: 1 главная Primary + 2 копии Secondary). Если сгорит один сервер, данные не пропадут.
- **2. Метаданные (Config Servers) -** Config Servers (Картотека кластера): служебные серверы, где хранится блокнот-карта. В ней записано, какой диапазон товаров на каком складе лежит.
- **3. Маршрутизатор (mongos) -** mongos (Охранник-навигатор): клиентское приложение общается только с ним. mongos не хранит данных на диске. Он принимает запрос 'дай товар ASUS', смотрит в блокнот Config Server, видит 'это лежит на Шарде 1', сам идет туда, забирает и возвращает клиенту. Программисту не нужно знать, где физически лежат серверы.
- **4. Чанги и балансировка -** Чанги (Chunks / Чанґи) и Ключ шардинга (Shard Key): коллекция нарезается на непрерывные блоки по ключу шардинга (Shard Key, например product_id). По умолчанию размер чанга равен 64 МБ. Когда чанг переполняется, он расщепляется (chunk split), а балансировщик (Balancer) автоматически переносит часть чангов на менее загруженные шарды.

4. Почему BSON, а не чистый JSON? И анатомия поля _id

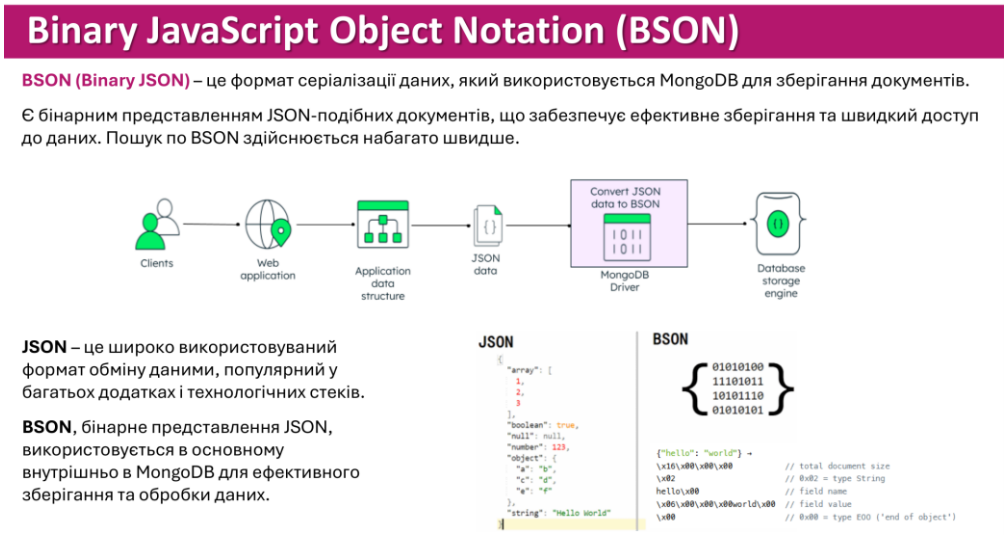


Рис. 3. Различие между форматами JSON (текст) и BSON (бинарный) (Слайд 18 лекции)

В чем разница и когда что применяется: JSON (JavaScript Object Notation) - это обычный текст. Человеку удобно читать и редактировать его глазами, а веб-сайтам - передавать его по интернету. Но если бы база данных хранила внутри себя чистый текст JSON, она бы дико тормозила: чтобы найти нужное поле в конце файла, пришлось бы парсить весь текст символ за символом.

Кто выполняет конвертацию: преобразование выполняет клиентский драйвер (например, pymongo в Python или драйвер Node.js внутри mongosh). Приложение отправляет JSON-объект, драйвер кодирует его в BSON и отсылает серверу mongod. При чтении драйвер декодирует BSON обратно в JSON.

BSON (Binary JSON) - это двоичный машинный формат. Два главных преимущества:

- **1. Быстродействие:** Скорость обхода полей: перед каждым элементом в BSON записана его длина в байтах. Сервер может мгновенно перепрыгнуть ненужный блок памяти, не читая его.
- **2. Типы данных:** Богатая типизация и экономия памяти: число 12345678 в тексте JSON занимает 8 байт (символов), а в BSON пакуется ровно в 4 байта. BSON добавляет точные даты (Date), целые числа 32 и 64 бит, сырые байты (BinData) и уникальный ключ ObjectId.

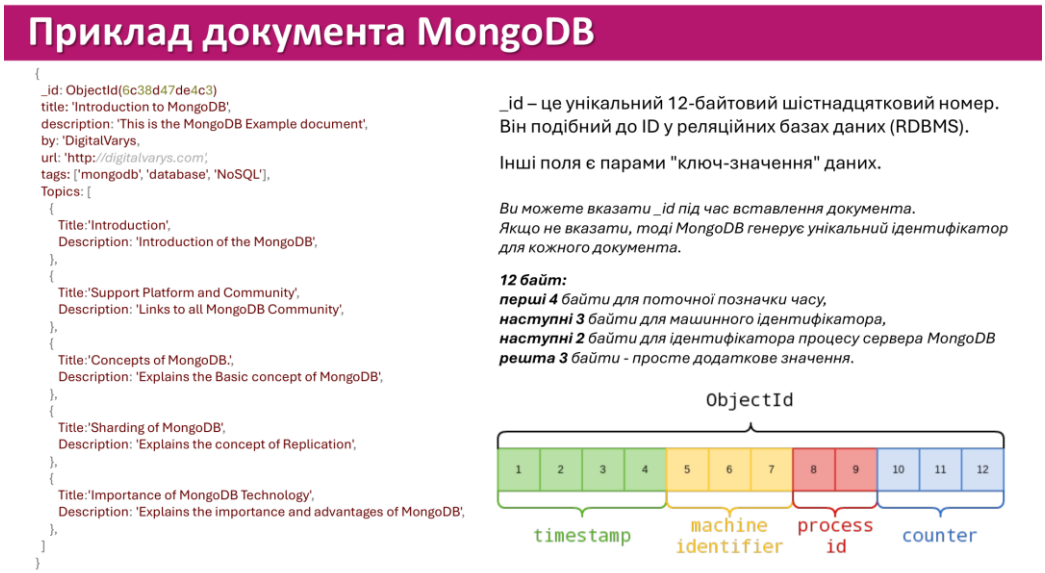


Рис. 4. Структура 12-байтного идентификатора документа ObjectId (Слайд 11 лекции)

Коронный капкан Жишковича: «Из чего состоит _id (ObjectId) на 12 байт?» (Слайд 11)
Поле _id занимает ровно 12 байт (в виде строки это 24 шестнадцатеричных символа). Формула 4-3-2-3:
• 4 байта - время создания документа (Unix timestamp в секундах). ВАЖНО: документы в MongoDB автоматически упорядочены по времени создания!
• 3 байта - уникальный идентификатор физического компьютера (хоста).
• 2 байта - идентификатор процесса mongod (PID сервера).
• 3 байта - инкрементный счетчик, который тикает при каждой новой вставке, исключая совпадения в одну секунду.
Ответ преподавателю: «12 байт: 4 часу, 3 комп'ютера, 2 процесу і 3 лічильника».

5. Графический клиент Compass и системные базы данных

При первом запуске MongoDB Compass и подключении по адресу mongodb://localhost:27017/ в навигационной панели отображаются 3 базы данных, созданные сервером по умолчанию:

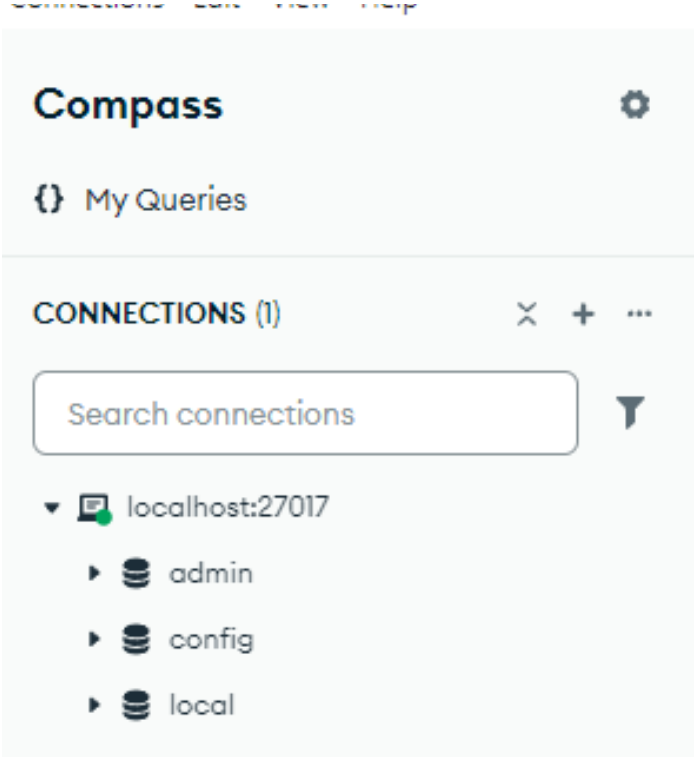


Рис. 5. Интерфейс MongoDB Compass и системные базы данных admin, config, local

- • admin: системная база для хранения учетных записей администраторов, паролей, ролей и прав доступа.
- • config: служебная база для распределенного шардинга (здесь хранятся настройки кластера и карта чанков).
- • local: журнал работы конкретного сервера (oplog). Данные из базы local НИКОГДА не реплицируются на другие серверы.

Зачем нужен Compass: это официальный графический интерфейс (GUI), позволяющий управлять базой без терминала - просматривать коллекции в виде дерева или таблицы, наглядно строить фильтры поиска, анализировать распределение типов полей (Schema Analysis) и отслеживать графики нагрузки сервера в реальном времени.

6. Разбор спроектированных документов интернет-магазина (TechStore)

По заданию требовалось спроектировать 2 коллекции с обязательным использованием двух ключевых механизмов NoSQL:

1. Вложенные документы (Embedded Documents / Денормализация)

В реляционных базах характеристики процессора (сокет, ядра, потоки, частоты, теплопакет) пришлось бы выносить в отдельную таблицу со связью по ID. В MongoDB мы вложили объект specifications прямо внутрь карточки товара PROD-102, а адрес доставки delivery_details - прямо внутрь заказа ORD-2026-002. Почему это круто: при открытии карточки товара база считывает весь объект за одно дисковое обращение без всяких JOIN.

2. Массивы (Arrays)

Массив тегов tags (['ips', '165hz', '2k_qhd']) в мониторе PROD-101 и массив купленных позиций items в заказе ORD-2026-001. В SQL для чека нужна связующая таблица Order_Items. В Mongo заказ содержит список купленных товаров прямо внутри себя.

> **Зачем нужна валидация синтаксиса в онлайн-валидаторе (Пункт 11 методички):**
> JSON подчиняется жесткому стандарту RFC 8259:
> 1. Все ключи ОБЯЗАНЫ быть в двойных кавычках "key" (одинарные кавычки 'key' вызовут ошибку синтаксиса).
> 2. Запятая после последнего элемента в объекте или массиве категорически запрещена (trailing comma).
> 3. Числа пишутся без кавычек, логические значения - строчными true/false.
> Зеленый статус VALID JSON в отчете для всех 6 документов подтверждает отсутствие синтаксических ошибок.

7. 3-Tier Code Anatomy: скрипт загрузки данных (main.py)

В репозитории подготовлен вспомогательный скрипт main.py для импорта JSON в базу. Ниже приведен его 3-уровневый разбор:

```
client = MongoClient('mongodb://localhost:27017/')
db = client['techstore']
db.products.drop()
res = db.products.insert_many(products)
```

- • Tier 1 (Назначение): подключение к локальному серверу MongoDB, переключение на базу techstore, очистка старой коллекции и пакетная загрузка подготовленных документов.
- • Tier 2 (Механика): MongoClient инициализирует пул соединений к порту 27017. db.products.drop() удаляет старую коллекцию вместе с индексами. insert_many() выполняет Bulk Insert за один сетевой запрос.
- • Tier 3 (Вопросы на защите): «Что вернет insert_many?» Ответ: объект InsertManyResult со списком сгенерированных ObjectId. «Что будет, если коллекция не существовала?» Ответ: Mongo создаст ее автоматически на лету (Lazy Creation).

8. Ответы на все 10 контрольных вопросов методички (Шпаргалка для сдачи)

> **1. Що таке документо-орієнтована БД і чим відрізняється від реляційної?**
> Це NoSQL СКБД, яка зберігає дані у вигляді самодостатніх ієрархічних документів (BSON/JSON) замість фіксованих таблиць. Відрізняється динамічною схемою (schemaless) та заміною важких з'єднань JOIN на природну вкладеність даних.

2. Яку NoSQL-модель даних використовує MongoDB?
Документну модель (Document Store).

3. Чим відрізняються документи MongoDB від рядків у реляційних БД?
Рядок у RDBMS є плоским і жорстко обмежений колонками таблиці. Документ MongoDB - це гнучке дерево пар 'ключ: значення', може містити вкладені об'єкти та списки (масиви), а сусідні документи можуть мати абсолютно різні набори полів.

4. Що таке 'динамічна схема' (Dynamic Schema / Schemaless) в MongoDB?
Це властивість бази зберігати документи довільної структури без попереднього оголошення колонок через CREATE TABLE. Структура інтерпретується програмою під час читання (Schema-on-Read).

5. Які основні версії MongoDB і в яких операційних системах вона може працювати?
Версії: Community Edition (відкрита безкоштовна), Enterprise Advanced (комерційна для бізнесу) та хмарна Atlas. Працює на Windows, Linux та macOS.

6. Назвіть ключові відмінності між Community та Enterprise версіями?
Enterprise має корпоративні модулі безпеки (LDAP, Kerberos, шифрування дисків), движок обчислень у пам'яті (In-Memory), детальний аудит дій та офіційну підтримку 24/7.

7. Яка різниця між виконуваними файлами mongod та mongosh?
mongod - це головний серверний процес (демон/служба), який керує файлами на диску, пам'яттю та портами. mongosh - це клієнтська консоль (термінал на Node.js), через яку користувач пише команди до бази.

8. Які функції при роботі з базами даних підтримує MongoDB Compass?
Перегляд і редагування документів, графічний конструктор фільтрів пошуку, аналіз схеми та типів полів, побудова конвеєрів агрегації та моніторинг навантаження сервера.

9. Поясніть, що таке об'єкт та масив у форматі JSON?
Об'єкт ({}) - це невпорядкований набір пар 'ключ: значення' у фігурних дужках. Масив ([]) - це впорядкований список елементів будь-якого типу через кому у квадратних дужках.

10. Що таке BSON і чому MongoDB використовує саме його, а не чистий JSON?
BSON - це двійкове представлення JSON. MongoDB використовує його заради високої швидкості (префікси довжини дозволяють миттєво пропускати поля при читанні) та розширених типів (Date, ObjectId, 64-бітні числа).

9. Вопросы-капканы преподавателя на защите

Капкан 1: «Що таке WiredTiger і навіщо він у MongoDB?»
WiredTiger - це стандартний внутрішній движок зберігання даних (Storage Engine) у MongoDB з версії 3.2. Він відповідає за запис файлів на диск, стиснення даних (Snappy/zlib, що економить до 70% місця) та керування кешем в RAM.

Капкан 2: «Що таке ледаче створення (Lazy Creation) колекцій?»
У MongoDB не обов'язково викликати команду створення колекції заздалегідь. Як тільки в базу вставляється перший документ, MongoDB автоматично створює і нову базу даних, і нову колекцію на льоту.

Капкан 3: «Чи є в MongoDB транзакції ACID?»
Традиційно в NoSQL атомарність гарантується на рівні одного документа (всі зміни всередині однієї карточки відбуваються надійно). Багатодокументні транзакції підтримуються починаючи з версії 4.0, але їх уникають за рахунок грамотної вкладеної структури.

Капкан 4: «Чому в замовленні ORD-2026-002 вказано місто Львів та Нова Пошта №42?»
Це персоналізація документа для підтвердження індивідуального виконання роботи: реальне місто навчання та ПІБ студента ФеП-33 Абдулханова Абдул-Рахіма.

10. Блиц-тренажер перед защитой (Active Recall)

Закрой рукой правый столбец и ответь на каждый вопрос за 5 секунд:

Вопрос преподавателя	Быстрый опорный ответ
Из чего состоит ObjectId на 12 байт?	Формула 4-3-2-3: 4 байта времени, 3 байта сервера, 2 байта процесса, 3 байта счетчика.
Чем отличается mongod от mongosh?	mongod - серверный демон; mongosh - консольный клиент.
Зачем денормализация в NoSQL?	Чтобы не делать медленные JOIN и забирать всю информацию за одно чтение.
Почему база хранит BSON, а не JSON?	BSON двоичный, быстрее читается за счет префиксов длины и умеет хранить Date и ObjectId.
Как работает шардинг?	mongos направляет запрос, Config Server хранит карту, Shards (Replica Sets) хранят куски данных.